QoreDB

LOCAL-FIRST DATABASE CLIENT

The SQL Performance Cheat Sheet

Find the slow query, read the plan, fix the index. A field guide to the handful of causes behind almost every slow query — and what to type to prove it, on each engine you actually use.

PostgreSQL · MySQL 8 · SQLite · Verified against the docs · Copy-paste ready

01 Measure, don't guess

The plan is the ground truth

Every optimisation starts the same way: **get the execution plan**. Intuition about which query is slow is wrong often enough that it isn't worth acting on. The plan tells you what the database actually did — not what you hoped it would do.

Get a plan

ENGINE	COMMAND	NOTES
POSTGRES	`EXPLAIN (ANALYZE, BUFFERS) <query>`	`ANALYZE` **really runs the query**. For writes, wrap in `BEGIN; … ROLLBACK;`. `BUFFERS` shows cache hits vs disk reads.
MYSQL	`EXPLAIN ANALYZE <query>`	MySQL 8.0.18+; also runs the query. Gives actual time and rows per iterator. Older servers: plain `EXPLAIN`. **MariaDB differs** — use `ANALYZE <statement>`.
SQLITE	`EXPLAIN QUERY PLAN <query>`	Describes the plan; does **not** execute it and prints **no costs or row counts**. Plain `EXPLAIN` dumps bytecode — not what you want.

ESTIMATED VS ACTUAL

`EXPLAIN` alone shows what the planner **expects**. `EXPLAIN ANALYZE` shows what **happened**. The gap between the two is the single most diagnostic number in the whole plan — see "red flag #1" below. Always reach for the ANALYZE form when the query is safe to run.

Reading a Postgres plan

Read it **inside-out and bottom-up**: the deepest, most-indented node runs first, and each node feeds its parent.

```
Nested Loop  (cost=0.43..41982.11 rows=1 width=64) (actual time=0.090..2451.876 rows=218450 loops=1)
  ->  Seq Scan on orders  (cost=0.00..41973.00 rows=1 width=32) (actual time=0.042..261.418 rows=218450 loops=1)
        Filter: (status = 'pending'::text)
        Rows Removed by Filter: 1781550
  ->  Index Scan using customers_pkey on customers  (cost=0.43..9.11 rows=1 width=36) (actual time=0.010..0.010 rows=1
loops=218450)
        Index Cond: (id = orders.customer_id)
Planning Time: 0.240 ms
Execution Time: 2461.063 ms
```

Diagnosis. The planner estimated `rows=1` from the `orders` scan; it really got **218 450**. Believing it would loop once, it chose a Nested Loop — and then ran the inner index scan **218 450 times**. **Follow the time, not the row counts:** inner-node times are per loop, so those lookups cost ≈ 218 450 × 0.010 ms ≈ **2.2 s of the 2.46 s**. The sequential scan your eye jumps to is only 261 ms — winning it back entirely would save 10 %.

The fix is the join, not an index. `ANALYZE orders;` corrects the estimate; the planner then sees 218 k rows and switches to a **Hash Join** — one pass over each table instead of 218 450 lookups. An index on `orders (status)` is not the answer: at 11 % of the table a plain index scan loses to a sequential one, and even if a bitmap scan won, it could only ever win back that 261 ms.

The five red flags

	WHAT YOU SEE	WHAT IT MEANS	FIRST THING TO TRY
1	Estimated rows off from actual by **>10×**	Stale or missing statistics. The planner is choosing from bad information, so every downstream choice is suspect. **Fix this before anything else.**	`ANALYZE tbl;` PG `ANALYZE TABLE tbl;` MY `ANALYZE;` LT
2	`Seq Scan` / `type: ALL` / `SCAN tbl` on a big table	Full table read. Correct and fast for small tables or when returning most rows — a problem when you wanted a handful of rows.	Index the filtered column; check the filter is sargable (§2)
3	High `loops=N` on an inner node; `Rows Removed by Filter` large	Work is being repeated per outer row, or read then discarded. Usually a knock-on effect of #1.	Fix stats; index the join/filter column
4	`Sort Method: external merge Disk: …` `Using filesort` / `Using temporary`	Sorting or grouping spilled out of memory onto disk.	Index matching the `ORDER BY`; or raise `work_mem` PG / `sort_buffer_size` MY

	WHAT YOU SEE	WHAT IT MEANS	FIRST THING TO TRY
5	`Buffers: … read=` large vs `hit=`	Reading from disk rather than cache. Either the working set doesn't fit in RAM, or you're touching far more data than needed.	Reduce rows touched first; only then size the cache (§4)

Join strategies — and when each is right

The planner picks these for you. You can't usually force them (and shouldn't) — but recognising **why** it picked wrong tells you what to fix.

STRATEGY	HOW IT WORKS	RIGHT WHEN	GOES WRONG WHEN
`Nested Loop`	For each row of the outer input, look up matches in the inner input.	The outer side is **small** and the inner side has an **index** on the join key.	The outer side turns out big. Cost is outer × inner-lookup, so a 1-row estimate that's really 100 k rows is catastrophic — **red flag #1**.
`Hash Join`	Build a hash table from the smaller input, then probe it with the larger one.	Joining **large, unindexed** inputs.	The build side doesn't fit in memory and spills to disk (**red flag #4**). Equality joins only on PG; MySQL 8.0.20+ also uses it for non-equi and outer joins.
`Merge Join`	Both inputs are sorted on the join key, then walked in lockstep.	Both inputs are **already sorted** — typically arriving straight from an index.	It has to sort them first, and the sort spills.

NOT EVERY ENGINE HAS ALL THREE

POSTGRES has all three and will choose freely. MYSQL has nested loop and hash join (8.0.18+; from 8.0.20 hash join fully replaced the old block nested loop) — but no merge join. SQLITE builds **every join as a nested loop**; when the inner table has no usable index it may materialise an **automatic index** on the fly, which the SQLite docs describe as essentially a hash join. Either way, an index on the join key matters more on SQLite than anywhere else.

MariaDB is not MySQL here. It has no MySQL-style hash join — only Block Nested Loop Hash, which is **off by default** (`join_cache_level < 3`).

Find the slow queries in the first place

POSTGRESQL

```postgresql
-- needs: shared_preload_libraries = 'pg_stat_statements'
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls, mean_exec_time, total_exec_time, query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

Sort by **total** time, not mean: a 5 ms query run 2 M times costs more than a 9 s report run nightly.

MYSQL

```mysql
SELECT digest_text, count_star,
       avg_timer_wait/1e9 AS avg_ms,
       sum_timer_wait/1e9 AS total_ms
FROM
performance_schema.events_statements_summary_by_digest
ORDER BY sum_timer_wait DESC
LIMIT 20;
```

Or enable the slow query log with `long_query_time = 1` and read it with `mysqldumpslow`.

Rule of thumb: **the fastest query is the one you never send.** Before tuning, ask whether the application needs those rows at all — caching, batching and pagination beat every index.

02 Indexes that actually get used

Order matters more than count

A B-tree index is a **sorted copy** of some columns plus a pointer back to the row. That single mental model explains everything below: you can seek to a prefix of the sort order, you can read ranges forward or backward, and you can't skip a leading column.

Composite index column order

An index on `(a, b, c)` serves `WHERE a`, `WHERE a AND b`, and `WHERE a AND b AND c`. It does **not** serve `WHERE b` alone — that's the **leftmost prefix rule**. One well-ordered composite index usually replaces three single-column ones.

THE ORDERING RULE: EQUALITY → SORT → RANGE

Put columns compared with = first, then the `ORDER BY` column, then columns compared with `< > BETWEEN LIKE 'x%'`. **A range column consumes the index:** nothing after it can be used for seeking.

WRONG ORDER — RANGE BEFORE SORT

```
-- query
WHERE tenant_id = ? AND created_at > ?
ORDER BY priority DESC

-- index (tenant_id, created_at, priority)
-- → seeks on tenant+created_at, then
--   sorts every match by priority
```

SORT BEFORE RANGE

```
-- same query
WHERE tenant_id = ? AND created_at > ?
ORDER BY priority DESC

-- index (tenant_id, priority, created_at)
-- → rows arrive already in priority
--   order; LIMIT can stop early
```

Which wins depends on selectivity — if `created_at > ?` eliminates 99.9 % of rows, the first index may still be better. **Measure both.** The rule is a starting hypothesis, not a law.

Covering indexes (index-only scans)

If the index contains **every column the query touches** — `SELECT` list, `WHERE`, `ORDER BY` — the engine never visits the table at all. The wider the table and the more rows returned, the bigger the win.

ENGINE	HOW	CONFIRM IN THE PLAN
POSTGRES	`CREATE INDEX ON orders (customer_id) INCLUDE (total);` INCLUDE stores the payload in leaf nodes only — it can't be searched or sorted on, but it keeps the index narrow. PG 11+.	`Index Only Scan` + `Heap Fetches: 0`
MYSQL	Add the columns to the index. Secondary indexes **already contain the primary key**, so `SELECT id` is free.	`Extra: Using index` (not "Using index condition")
SQLITE	Add the columns to the index.	`USING COVERING INDEX …`

POSTGRES GOTCHA: HEAP FETCHES

An index-only scan still checks the **visibility map** to know whether a row is visible to your transaction. On a table with recent writes those bits aren't set, so Postgres falls back to reading the heap and you lose the benefit — visible as a non-zero `Heap Fetches`. `VACUUM` sets those bits. A freshly bulk-loaded table that has never been vacuumed will quietly not use index-only scans.

What silently kills a seek (sargability)

An index works on the **bare column**. Wrap it in anything and the sort order no longer applies, so the engine scans.

THIS SCANS	THIS SEEKS	WHY
`WHERE YEAR(created_at) = 2025`	`WHERE created_at >= '2025-01-01'` `AND created_at < '2026-01-01'`	Function on the column. Half-open range also handles time components correctly.

THIS SCANS	THIS SEEKS	WHY
`WHERE LOWER(email) = ?`	`CREATE INDEX ON users (LOWER(email));` PG LT `CREATE INDEX i ON users ((LOWER(email)));` MY	Index the expression. MySQL: 8.0.13+, needs a name and **double parens**; its default collation is already case-insensitive, so you may not need it.
`WHERE name LIKE '%acme%'`	`pg_trgm` GIN index PG, or full-text search	A leading wildcard has no prefix to seek to. `LIKE 'acme%'` is fine.
`WHERE user_id = '42'` (int column)	`WHERE user_id = 42`	Implicit cast. In MySQL a string-vs-number comparison can force a full scan and, worse, compare unexpectedly.
`WHERE status != 'done'`	`WHERE status IN ('new','running')`	Negation matches most rows, so a scan is genuinely cheaper. Enumerate what you want instead.

`WHERE a = ? OR b = ?`	`SELECT … WHERE a = ?`
	`UNION ALL`
	`SELECT … WHERE b = ?`
	`  AND (a <> ? OR a IS NULL)`
		Two different columns can't share one seek. The second branch needs that guard or you get duplicates — and lose NULL rows. **See §3.**

When B-tree is the wrong index

B-tree is the default and the right answer ~90 % of the time. The exceptions are worth knowing, because no amount of column reordering fixes them.

TYPE	USE IT FOR	EXAMPLE	ENGINE
GIN	Many values inside one column: `jsonb` keys, arrays, full-text, and substring search via `pg_trgm`.	`CREATE INDEX ON docs` `USING GIN (body jsonb_path_ops);`	POSTGRES
BRIN	Huge **naturally ordered** append-only tables (time series). Default opclass stores min/max per block range — a tiny index, but it degrades as the column decorrelates from physical order. (`minmax_multi` and `bloom` opclasses handle outliers better.)	`CREATE INDEX ON events` `USING BRIN (created_at);`	POSTGRES
GiST	Ranges, geometry, nearest-neighbour, exclusion constraints.	`EXCLUDE USING GIST` `(room WITH =, during WITH &&)`	POSTGRES
Full-text	Word search. Don't fake it with `LIKE '%word%'`.	`tsvector` + GIN PG · `FULLTEXT` MY · FTS5 LT	ALL

Indexes are not free

Every index is extra work on `INSERT`, `UPDATE` and `DELETE`, extra disk, and extra memory competing for cache. Two indexes with the same leading columns are usually one index too many.

Check uptime before trusting `idx_scan = 0` — stats reset on `pg_stat_reset()`, and a quarterly report's index looks unused in May.

```
-- unused indexes, biggest first (PG)
SELECT relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

03 Query patterns worth knowing

Same rows, a fraction of the work

N+1 — the most expensive bug in ORMs

One query to fetch the list, then one query per item. Each is 1 ms and indexed, so nothing shows up in the slow query log — yet the endpoint takes 800 ms, almost all of it round-trip latency.

1 + 200 QUERIES

```
SELECT * FROM posts LIMIT 200;
-- then, per post, in application code:
SELECT * FROM users WHERE id = ?;
```

2 QUERIES

```
SELECT * FROM posts LIMIT 200;
SELECT * FROM users WHERE id = ANY(?);
-- or a single JOIN
```

Most ORMs have a name for the fix: `includes` (Rails), `selectinload` (SQLAlchemy), `Include` (EF Core), `with` (Laravel), `prefetch_related` (Django). **Tell:** the same query shape repeated with different parameters in your query log.

OFFSET pagination collapses at depth

`OFFSET 100000` makes the database produce 100 020 rows in order and throw away 100 000 of them. Page 1 is instant; page 5 000 is a table scan. Cost grows linearly with page number.

O(OFFSET) — SLOWER EVERY PAGE

```
SELECT id, title, created_at
FROM events
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 100000;
```

O(1) — KEYSET / "SEEK" PAGINATION

```
SELECT id, title, created_at
FROM events
WHERE (created_at, id) < (?, ?)   -- last row of prev page
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Needs an index on `(created_at DESC, id DESC)` and a **tiebreaker column** — without `id`, rows sharing a timestamp get skipped or duplicated across pages. Trade-off: no "jump to page 47", which is why infinite scroll and "load more" are its natural UIs.

ROW VALUES: EXPLAIN BEFORE YOU RELY ON THIS

The `(a, b) < (x, y)` form is standard SQL and seeks properly on POSTGRES and SQLITE 3.15+. MYSQL parses it, but its documented row-constructor range optimisation covers `IN()` only. Portable equivalent: `WHERE created_at < ? OR (created_at = ? AND id < ?)`.

OR across columns → UNION ALL

```sql
-- one seek is impossible: scan
SELECT * FROM t WHERE a = ? OR b = ?;

-- two seeks, indexes on (a) and (b)
SELECT * FROM t WHERE a = ?
UNION ALL
SELECT * FROM t
WHERE b = ? AND (a <> ? OR a IS NULL);
```

DON'T DROP THE IS NULL

The second branch must exclude rows the first already returned — but a bare `a <> ?` evaluates to **NULL, not true**, when `a IS NULL`. Without the `OR a IS NULL` you **silently lose** every row where `a` is NULL and `b` matches. On Postgres, `a IS DISTINCT FROM ?` says the same thing more safely.

This avoids the sort that `UNION` (distinct) would add. Postgres can sometimes do it for you via a BitmapOr; MySQL's index_merge is less reliable.

SELECT * is not free

- Wide columns (JSON, TEXT, BLOB) get read and sent for nothing — and in Postgres may mean extra TOAST fetches.
- It defeats **covering indexes**: one unneeded column forces a table lookup per row.
- It breaks silently when someone adds a column.

Counting large tables

`COUNT(*)` is exact and **O(n)**: MVCC means neither engine keeps a live row count. InnoDB traverses the smallest available secondary index; Postgres usually just scans the table (in parallel if it can). On 100 M rows that's seconds — per page load.

```sql
-- good enough for "About 4,300,000 results"
SELECT reltuples::bigint FROM pg_class
WHERE oid = 'public.events'::regclass;   -- PG

-- exact count of a bounded window
SELECT count(*) FROM (
  SELECT 1 FROM events WHERE … LIMIT 1001
) s;  -- "1000+" — cheap and honest
```

Use `::regclass` — table names repeat across schemas. `reltuples` is maintained by VACUUM/ANALYZE and is **-1 on a never-vacuumed table** (PG 14+), so handle that before showing it to a user. For exact live counts, keep a trigger-maintained counter — and accept the write contention.

DISTINCT is often a symptom

Reaching for `DISTINCT` to remove duplicates usually means a join fanned out rows. It hides the bug behind a sort of the whole result. Fix the join, or use `EXISTS`:

```sql
SELECT u.* FROM users u
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.user_id = u.id);
-- stops at the first match per user
```

Batch your writes. 10 000 single-row `INSERT`s in autocommit means 10 000 separate transactions, each with its own commit and round-trip. Wrapping them in **one transaction** — or one multi-row `INSERT` — is routinely **orders of magnitude faster**. It's the single biggest win available on SQLite, and a large one everywhere else.

04 Engine-specific tuning

Where the engines stop agreeing

POSTGRESQL

KNOB	WHAT TO KNOW
`work_mem`	**Per sort/hash node, per worker** — not per query, not per connection. A query with 4 sorts across 8 parallel workers can use many multiples. Hash nodes get `work_mem × hash_mem_multiplier` (default **2.0** since PG 15). Raise it per-session for a big report, not globally.
`shared_buffers`	Commonly ~25 % of RAM. Postgres also benefits from the OS page cache, so more is not automatically better.
`effective_cache_size`	Not an allocation — a **hint** about the cache available to a **single query**, so divide by expected concurrency. Too low and the planner avoids index scans.
`random_page_cost`	Default **4.0**, which already assumes most random reads are **cached** — not that you're on a spinning disk. Lower it (≈`1.1`) when random reads really are nearly as cheap as sequential; **raise** it for magnetic disks.
`autovacuum`	Reclaims dead rows and sets visibility-map bits. Bulk loads outpace it — run `ANALYZE` manually after one. Don't disable it unless your workload is extremely predictable (and anti-wraparound vacuum runs regardless).

BLOAT & LONG TRANSACTIONS

An `UPDATE` writes a new row version; the old one lingers until vacuumed. Vacuum still runs while a connection sits **idle in transaction** — but it **can't remove rows still visible to that old snapshot**, so dead rows accumulate and scans get slower for everyone. Check `pg_stat_activity` for `state = 'idle in transaction'` and set `idle_in_transaction_session_timeout`.

LIKE 'foo%' not using the index? In any non-C collation you need `CREATE INDEX ON t (col text_pattern_ops);` — the default operator class sorts by locale rules, which don't match character-by-character prefix comparison. That index won't serve ordinary `<` `>` comparisons, so you may need both.

BUILD WITHOUT LOCKING

`CREATE INDEX CONCURRENTLY …` avoids holding a write lock against a live table. It's slower, can't run inside a transaction, and can leave an `INVALID` index behind if it fails — check for that, drop it, retry.

MYSQL / INNODB

EVERYTHING FOLLOWS FROM THE CLUSTERED PK

InnoDB stores the **table itself** in primary-key order, and every secondary index stores the primary key as its row pointer. Two consequences:

- **A fat PK bloats every secondary index.** A 36-char UUID string PK across 6 indexes wastes gigabytes versus a `BIGINT`.
- **A random PK fragments writes.** UUIDv4 inserts land in random pages, causing page splits and random I/O. Use `AUTO_INCREMENT`, or time-ordered **UUIDv7 / ULID** if you need opaque IDs.

KNOB	WHAT TO KNOW
`innodb_buffer_pool_size`	The **one setting that matters most**. Caches data and indexes. ~70 % of RAM on a dedicated server; the default is far too small.
`innodb_flush_log_at_trx_commit`	`1` = fully durable (default). `2` = survives a process crash but not power loss. Only relax it if you can afford to lose up to a second of commits — and note the once-per-second flush is **not guaranteed**.
Prefix indexes	`INDEX (url(64))` keeps long-string indexes small — but it stores only the first 64 chars, so it **can't cover a query that needs the full value** and can't fully resolve `ORDER BY`.

Read `EXPLAIN` right: `type: ALL` = full scan, `index` = full index scan (still every row!), `range`/`ref`/`eq_ref`/`const` = progressively better. Compare `rows` examined against rows returned — a big ratio is the whole problem in one number.

Partial indexes PG SQLITE

Index only the rows you actually query. A job queue where `status='pending'` is 0.1 % of a 50 M row table gets an index **orders of magnitude smaller** — it stays in cache, and the other 99.9 % of rows cost nothing to maintain.

```
CREATE INDEX jobs_pending_idx ON jobs (created_at)
  WHERE status = 'pending';
```

The planner only uses it when the query's `WHERE` clause provably implies the index predicate. **MySQL has no partial indexes** — use a generated column, or an index on `(status, created_at)`.

The 10-minute triage

In order. Most slow queries are fixed by step 3.

- [ ] **Reproduce with numbers.** Get the actual query and its parameters. "The dashboard is slow" is not a query.
- [ ] **Get the plan.** `EXPLAIN ANALYZE`. Read bottom-up.
- [ ] **Compare estimated vs actual rows.** Off by >10×? Run `ANALYZE` and re-plan before touching anything else.
- [ ] **Follow the time, not the rows.** Which node actually spends it? Watch for per-loop costs × a big `loops=`.
- [ ] **Check sargability.** Is the column wrapped in a function or cast? (§2)
- [ ] **Order the index: equality → sort → range.** Try to make it covering.
- [ ] **Re-measure.** Same plan? The index isn't usable — find out why rather than adding another.
- [ ] **Only now touch config.** Memory settings fix a class of queries; they rarely fix this one.

SQLITE

SQLite's defaults are tuned for **maximum compatibility and safety**, not speed. A handful of pragmas at connection time typically buys an order of magnitude.

```
PRAGMA journal_mode = WAL;     -- once per DB, persists
PRAGMA synchronous = NORMAL;   -- per connection
PRAGMA busy_timeout = 5000;    -- ms; default 0 (!)
PRAGMA foreign_keys = ON;      -- OFF by default (!)
PRAGMA cache_size = -64000;    -- negative = KiB (~64 MB)
ANALYZE;                       -- planner needs stats
```

PRAGMA	WHY
`journal_mode=WAL`	Readers stop blocking the writer and vice versa. QoreDB opens SQLite in WAL by default. Adds `-wal`/`-shm` sidecar files — don't delete them.
`busy_timeout`	Default **0** means a concurrent writer fails instantly with `SQLITE_BUSY` instead of waiting. Most "SQLite can't handle concurrency" reports are this.
`PRAGMA optimize`	Refreshes stats cheaply. **Short-lived** connections: run it just before closing. **Long-lived**: run `PRAGMA optimize=0x10002;` on open, then `PRAGMA optimize;` periodically.

SYNCHRONOUS = NORMAL: READ THIS BEFORE SHIPPING

In WAL mode, `NORMAL` **will not corrupt your database** — integrity is guaranteed. But it **does give up durability**: a power loss or OS crash can roll back recent committed transactions. Fine for caches, derived data and dev. For a system of record where a lost commit matters, keep `FULL`.

One writer at a time, always. If two processes must write concurrently and `busy_timeout` isn't enough, SQLite is the wrong tool — not a tuning problem.

DO THIS IN QOREDB

Every technique in this guide works in any client — that's the point. QoreDB just removes the tab-switching: **execution plan visualisation**, one editor across PostgreSQL, MySQL, SQL Server, SQLite, DuckDB, MongoDB, Redis, CockroachDB and ClickHouse, and **notebooks** (`.qnb`) that mix SQL and Markdown — so a triage like the one above becomes a runbook you can re-run and share instead of a scrollback you lose.

Verify against your version. Behaviour and defaults change between releases. Primary sources: `postgresql.org/docs` · `dev.mysql.com/doc` · `sqlite.org/docs.html`.